SOURCE CODE SUBMISSION

Air Pollution Monitoring System

Complete ESP32 Embedded C++ Firmware Implementation (MQ-135, DHT11 & MQTT)

AUTHOR	Giri G	**DOMAIN**	Embedded Systems & IoT
FILE NAME	AirPollutionMonitoring.ino	**DATE**	17/10/2026
ORGANIZED BY	upskill Campus (USC) & The IoT Academy	**PARTNER**	UniConverge Technologies Pvt Ltd (UCT)

Hardware mapping

MQ-135 analog output: GPIO 34; DHT11 data: GPIO 15; optional buzzer: GPIO 18. Check the sensor module's voltage before connecting its analog output to the ESP32.

Firmware source

```
001 /*
002  * Air Pollution Monitoring System
003  * ESP32 + MQ-135 + DHT11 + Wi-Fi/MQTT
004  *
005  * Libraries:
006  *   - PubSubClient by Nick O'Leary
007  *   - DHT sensor library by Adafruit
008  *   - Adafruit Unified Sensor
009  *
010  * Hardware:
011  *   MQ-135 analog output -> ESP32 GPIO 34
012  *   DHT11 data           -> ESP32 GPIO 15
013  *   Active buzzer (+)    -> ESP32 GPIO 18 (optional; use a suitable driver if needed)
014  *
015  * IMPORTANT:
016  * MQ-135 is a broad gas/air-quality sensor. Raw ADC values are not certified AQI
017  * or ppm values. Calibrate the sensor for the target gas and environment before
018  * making quantitative claims. Warm-up time is required.
019  */
020
021 #include <WiFi.h>
022 #include <PubSubClient.h>
023 #include "DHT.h"
024
025 #define MQ135_PIN 34
026 #define DHT_PIN 15
027 #define DHT_TYPE DHT11
028 #define BUZZER_PIN 18
029
030 const char* WIFI_SSID = "YOUR_WIFI_SSID";
031 const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
032 const char* MQTT_SERVER = "broker.hivemq.com";
033 const int MQTT_PORT = 1883;
034
035 // Example threshold only: tune after observing/calibrating your own sensor.
036 const int AIR_QUALITY_ALERT_THRESHOLD = 1800;
037 const unsigned long PUBLISH_INTERVAL_MS = 5000;
038
039 WiFiClient wifiClient;
040 PubSubClient mqttClient(wifiClient);
041 DHT dht(DHT_PIN, DHT_TYPE);
042
043 unsigned long lastPublish = 0;
044
045 void connectWiFi() {
046   Serial.print("Connecting to Wi-Fi");
047   WiFi.mode(WIFI_STA);
048   WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
049
```

```
050   while (WiFi.status() != WL_CONNECTED) {
051     delay(500);
052     Serial.print(".");
053   }
054
055   Serial.println();
056   Serial.print("Wi-Fi connected. IP address: ");
057   Serial.println(WiFi.localIP());
058 }
059
060 void connectMQTT() {
061   while (!mqttClient.connected()) {
062     Serial.print("Connecting to MQTT broker... ");
```

```
    String clientId = "AirPollutionESP32-" + String((uint32_t)ESP.getEfuseMac(), HEX);

    if (mqttClient.connect(clientId.c_str())) {
      Serial.println("connected");
    } else {
      Serial.print("failed, state=");
      Serial.print(mqttClient.state());
      Serial.println("; retrying in 5 seconds");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(MQ135_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  dht.begin();
  connectWiFi();

  mqttClient.setServer(MQTT_SERVER, MQTT_PORT);
  mqttClient.setBufferSize(256);
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }

  if (!mqttClient.connected()) {
    connectMQTT();
  }
  mqttClient.loop();

  unsigned long now = millis();
  if (now - lastPublish >= PUBLISH_INTERVAL_MS) {
    lastPublish = now;

    int airRaw = analogRead(MQ135_PIN);
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    bool airAlert = (airRaw >= AIR_QUALITY_ALERT_THRESHOLD);
    digitalWrite(BUZZER_PIN, airAlert ? HIGH : LOW);

    Serial.print("MQ-135 raw ADC: ");
    Serial.println(airRaw);

    if (!isnan(temperature) && !isnan(humidity)) {
      Serial.print("Temperature (C): ");
      Serial.println(temperature, 1);
      Serial.print("Humidity (%): ");
      Serial.println(humidity, 1);

      char tempPayload[16];
      char humidityPayload[16];
      dtostrf(temperature, 1, 1, tempPayload);
      dtostrf(humidity, 1, 1, humidityPayload);

      mqttClient.publish("airpollution/temperature_c", tempPayload);
```

```
      mqttClient.publish("airpollution/humidity_percent", humidityPayload);
    } else {
      Serial.println("DHT11 read failed; check wiring and sensor.");
    }

    char airPayload[12];
    itoa(airRaw, airPayload, 10);
    mqttClient.publish("airpollution/mq135_raw_adc", airPayload);
    mqttClient.publish("airpollution/alert", airAlert ? "HIGH" : "NORMAL");

    Serial.print("Air-quality status: ");
    Serial.println(airAlert ? "HIGH (threshold exceeded)" : "NORMAL");
  }
}
```